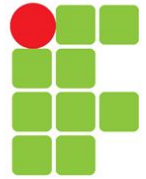


INSTITUTO FEDERAL
SÃO PAULO



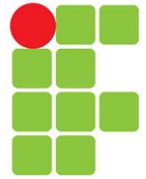
HTTP e Arquitetura REST para APIs Web

Desenvolvimento Web Backend (BRADWBK)

Prof. Luiz Gustavo Diniz de Oliveira Véras

E-mail: gustavo_veras@ifsp.edu.br

Objetivos



✔REST

✔HTTP

- Recursos
- Verbos
- Status

✔Arquitetura Restful

✔Como definir uma API REST?

✔Recursos do Framework Spring para APIs REST

✔Criando WebServices com Spring MVC

✔Recursos HTTP no Spring MVC

APIs WEB

Definimos inicialmente o paradigma de API necessário para o desenvolvimento da nossa aplicação. Na aula de hoje seguiremos utilizando o REST (*Representational State Tranfer*).

APIs baseadas em Request-Response (HTTP)

- **REST**
- RPC (Remote Procedure Call)
- GraphQL

APIs orientadas à eventos

- WebHooks
- WebSockets
- HTTP Streaming

REST

REST (Representational State Transfer) refere-se a um grupo de restrições de design dentro da arquitetura de software que geram sistemas distribuídos eficientes, confiáveis e escaláveis. Um sistema é denominado RESTful quando adere a todas essas restrições.



REST

REST significa *“Representational State Transfer*", que em português significa transferência de estado representacional. Essa arquitetura foi criada pelo cientista da computação **Roy Fielding,** que a apresentou pela primeira vez em 2000 em sua famosa dissertação de doutorado.

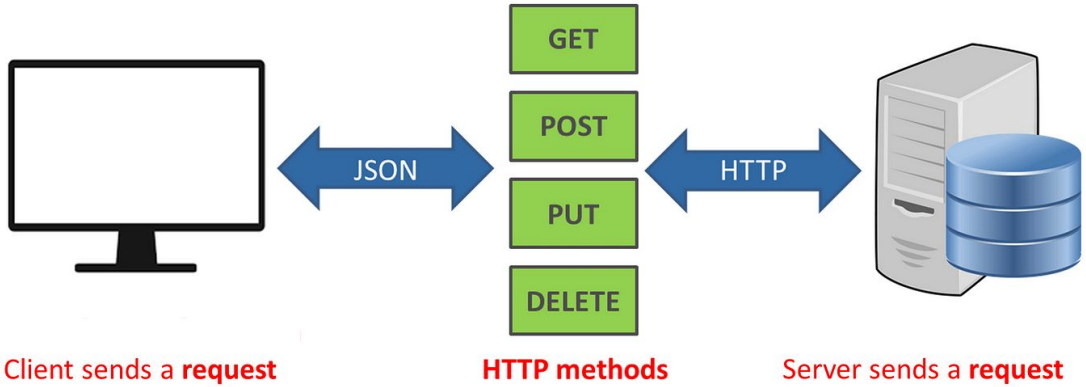
REST é uma maneira simples de organizar interações em sistemas independentes e a forma mais popular atualmente de desenvolver APIs Web.

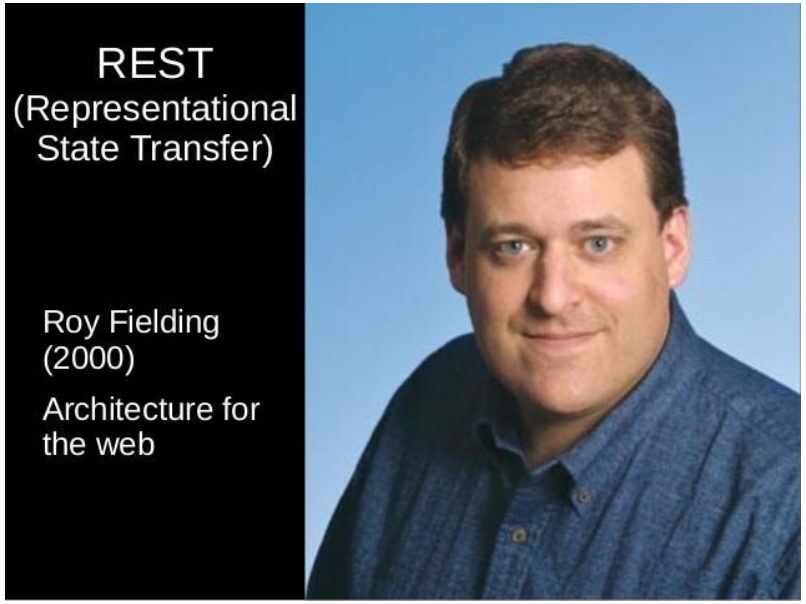


Roy Fielding

REST (*Representational State Transfer*)

Como a implementação de REST mais conhecida para Web envolve o **HTTP** e o utiliza como base, é importante primeiro entendermos como o **HTTP** de fato funciona antes de apresentá-lo.





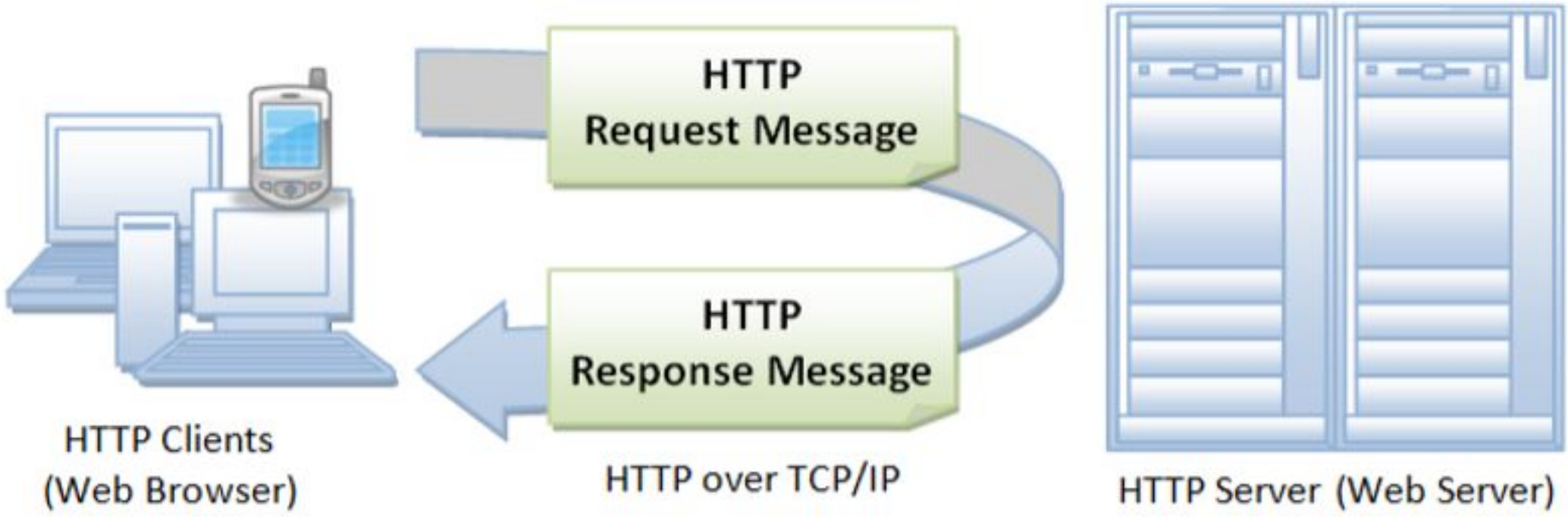
Protocolo HTTP

O HTTP (*Hypertext Transfer Protocol*) é o principal protocolo utilizado na WWW (*World Wide Web*).

É através dele que um **Client** (web browser, outro servidor, etc) faz comunicação com um **Server** (um computador com um programa que "escuta" solicitações de **recursos**) através de um modelo de **requisição/resposta (request/response)**.

Protocolo HTTP

Modelo Request/Response do HTTP



Recursos na Web

O alvo de uma requisição HTTP é chamada de "**resource**", ou **recurso** em português, com a natureza ainda não definida. Isso significa que um **recurso** pode ser:

- um documento
- uma foto
- qualquer outra coisa



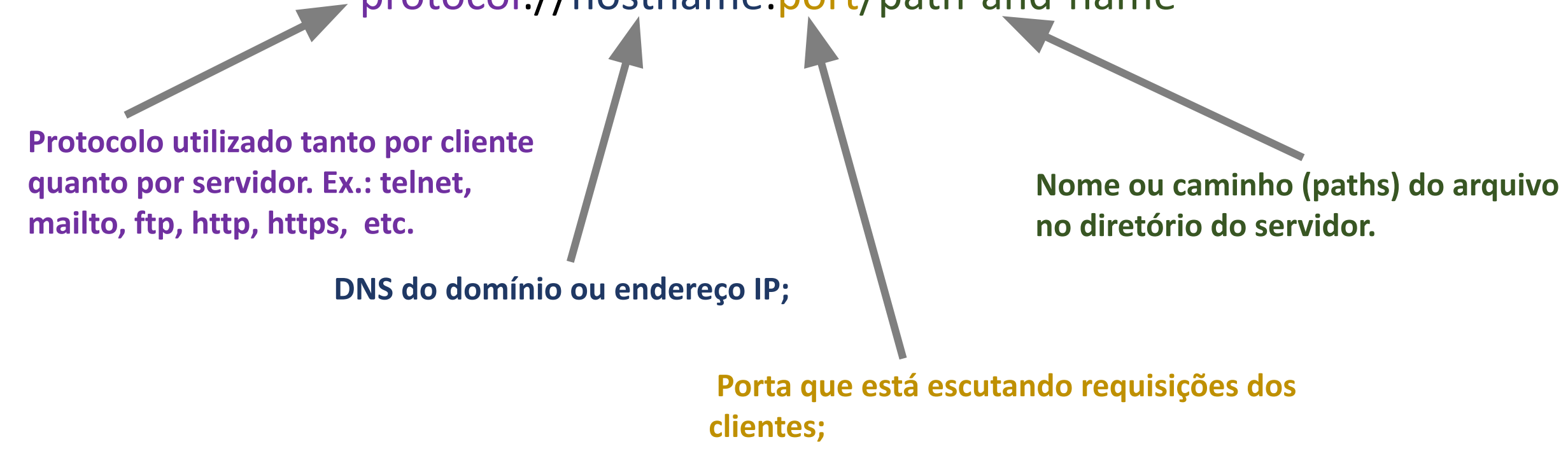
Cada recurso é identificado por uma **Localização de recursos uniforme**, do inglês **Uniform Resource Locator (URL)** usada pelo HTTP para identificar recursos. **URLs são conhecidos como endereços da Web.**

Recursos na Web

Um recurso é qualquer **porção de informação** identificado por uma URL.

URL - Uniform Resource Location

protocol://hostname:port/path-and-name

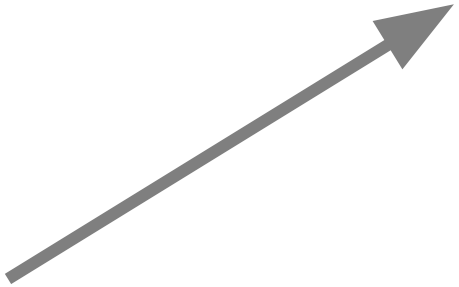


Recursos na Web

Um recurso é qualquer **porção de informação** identificado por uma URL.

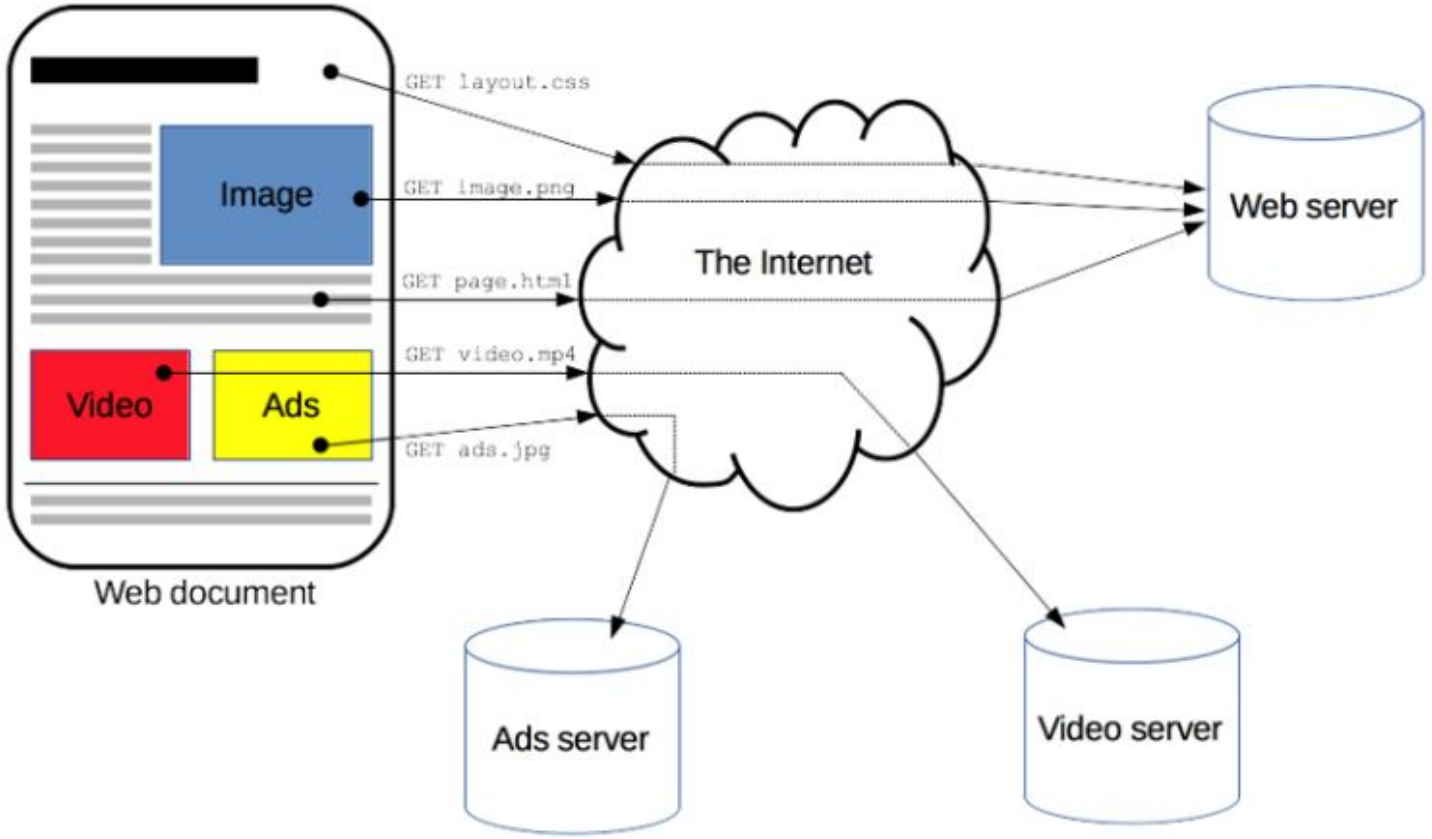
URL - Uniform Resource Location

protocol://hostname:port/path-and-name?key1=value1&key2=value2



Querys (ou parâmetros) são uma lista de pares chaves/valores separados com o símbolo &. O servidor web pode usar esses parâmetros para fazer coisas extras depois retornando o recurso para o usuário.

Recursos na Web



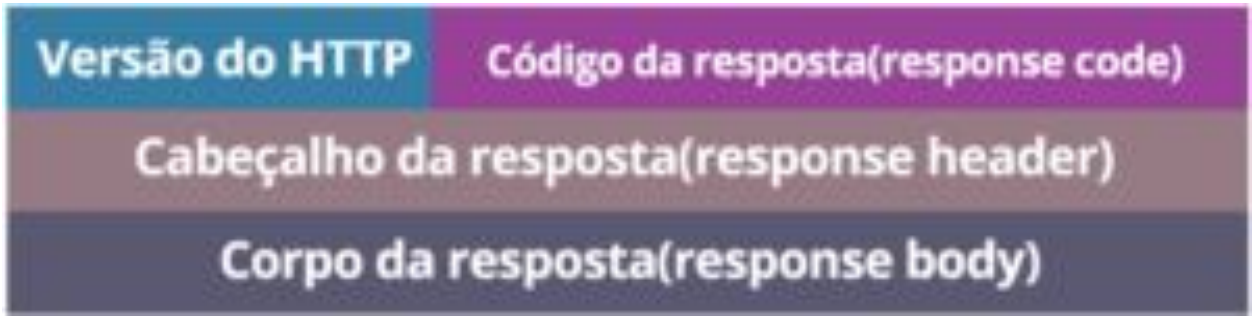
Um documento HTML completo é construído a partir de diferentes sub-documentos solicitados via HTTP, como por exemplo texto, descrição do layout, imagens, vídeos, scripts e muito mais.

Protocolo HTTP

Uma requisição HTTP tem o seguinte formato:



E uma resposta HTTP:



Protocolo HTTP

Exemplo:

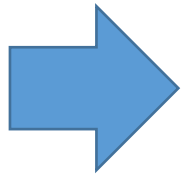
Em uma requisição HTTP a URL é convertida para um formato próprio.

```
http://www.meudominio.com:8080/docs/index.html
```



```
GET /docs/index.html HTTP/1.1
Host: www.meudominio.com:8080
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
(blank line)
```

HTTP Request





HTTP Server

Protocolo HTTP

Exemplo:

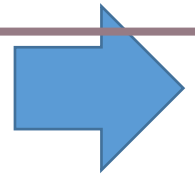
Em uma requisição HTTP a URL é convertida para um formato próprio.

```
http://www.meudominio.com:8080/doc/index.html
```



HTTP Request

```
GET /docs/index.html HTTP/1.1
Host: www.meudominio.com:8080
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
(blank line)
```





HTTP Server

Protocolo HTTP

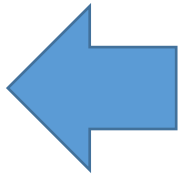
Exemplo:

O servidor interpreta a requisição e devolve o conteúdo solicitado com uma resposta.

```
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2009 08:56:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Sat, 20 Nov 2004 07:16:26 GMT
ETag: "10000000565a5-2c-3e94b66c2e680"
Accept-Ranges: bytes
Content-Length: 44
Connection: close
Content-Type: text/html
X-Pad: avoid browser bug

<html><body><h1>It works!</h1></body></html>
```

HTTP Response





HTTP Server

Protocolo HTTP

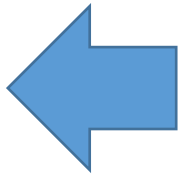
Exemplo:

O servidor interpreta a requisição e devolve o conteúdo solicitado.

```
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2009 08:56:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Sat, 20 Nov 2004 07:16:26 GMT
ETag: "10000000565a5-2c-3e94b66c2e680"
Accept-Ranges: bytes
Content-Length: 44
Connection: close
Content-Type: text/html
X-Pad: avoid browser bug

<html><body><h1>It works!</h1></body></html>
```

HTTP Response





HTTP Server

Protocolo HTTP

Exemplo:

O servidor interpreta a requisição e devolve o conteúdo solicitado.

```
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2009 08:56:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Sat, 20 Nov 2004 07:16:26 GMT
ETag: "10000000565a5-2c-3e94b66c2e680"
Accept-Ranges: bytes
Content-Length: 44
Connection: close
Content-Type: text/html
X-Pad: avoid browser bug

<html><body><h1>It works!</h1></body></html>
```

HTTP Response



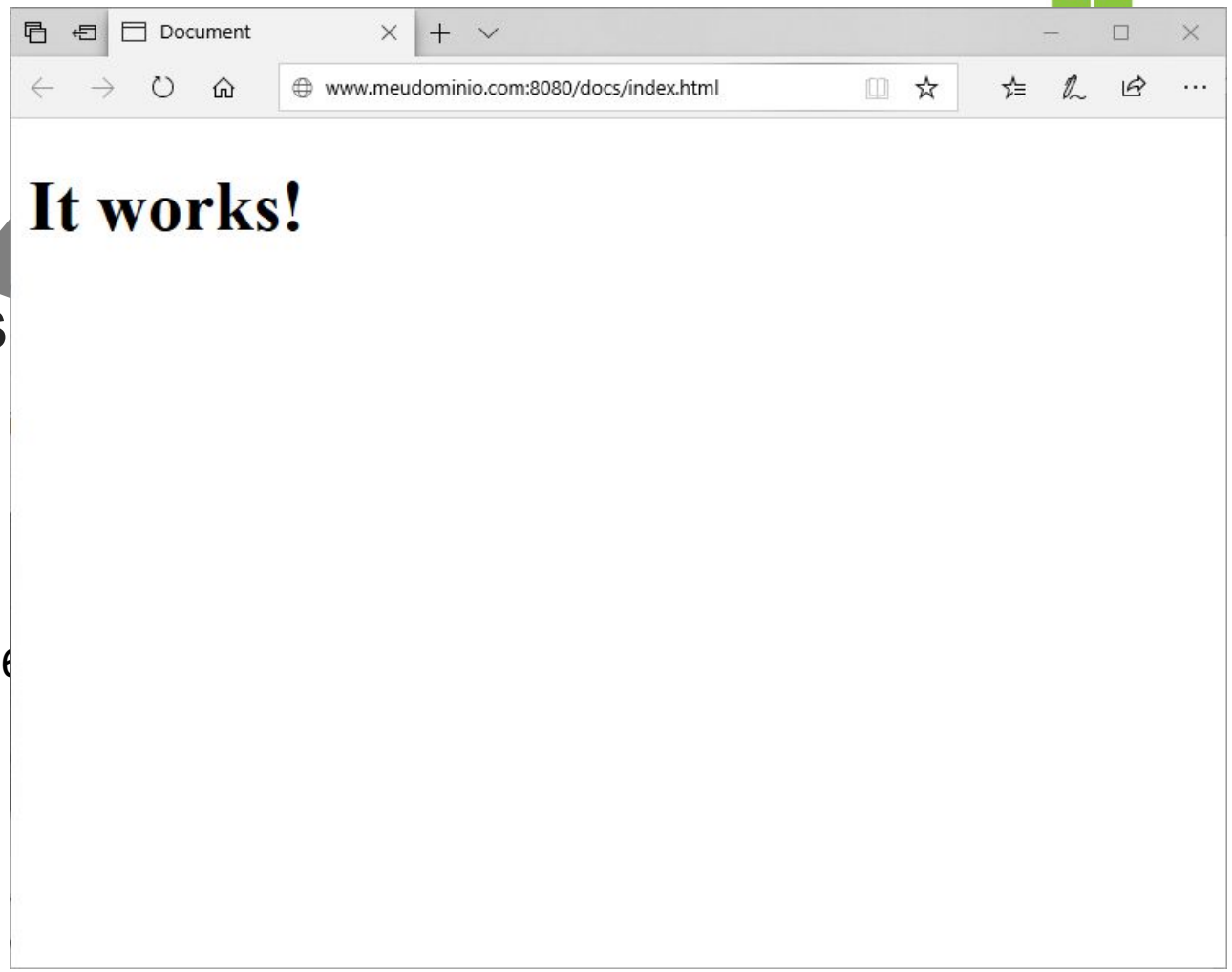
HTTP Server

Existe uma linha em branco que separa o header do body da resposta.

Protocolo HTTP

Exemplo:

O servidor interpreta a requis
solicitado.



HTTP Server

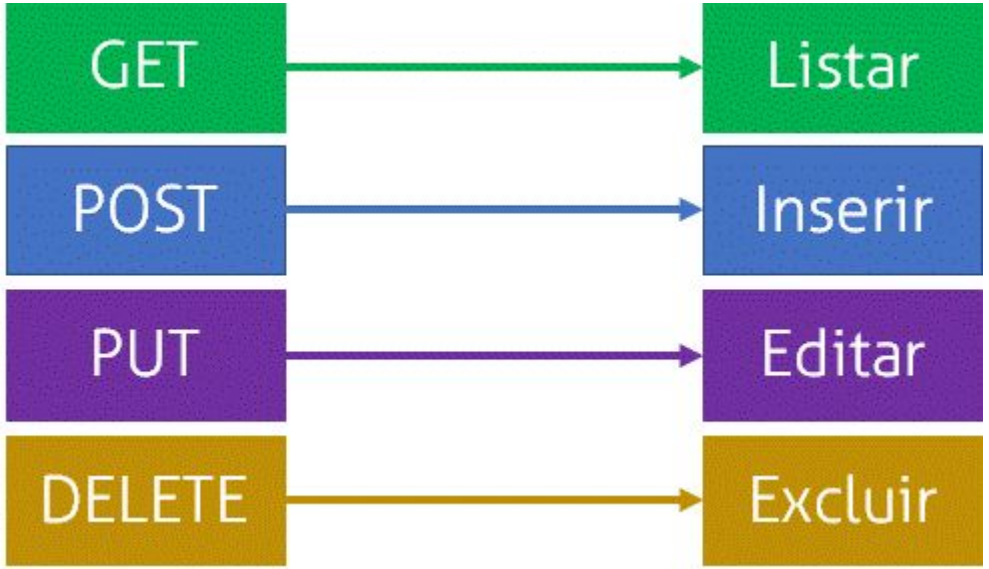
```
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2009 08:56:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Sat, 20 Nov 2004 07:16:26
ETag: "10000000565a5-2c-3e94b66c2e680"
Accept-Ranges: bytes
Content-Length: 44
Connection: close
Content-Type: text/html
X-Pad: avoid browser bug

<html><body><h1>It works!</h1></body></html>
```

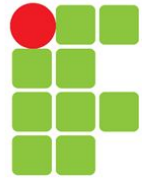
Verbos HTTP

Os verbos permitem indicar a um servidor que tipo de operação um Navegador (ou outro software) gostaria de solicitar na comunicação pela internet.

É importante conhecê-los para indicar operações ao servidor na requisição HTTP e interpretar o resultado da solicitação.



Verbos HTTP



Principais

GET

O método `GET` solicita a representação de um recurso específico. Requisições utilizando o método `GET` devem retornar apenas dados.

POST

O método `POST` é utilizado para submeter uma entidade a um recurso específico, frequentemente causando uma mudança no estado do recurso ou efeitos colaterais no servidor.

PUT

O método `PUT` substitui todas as atuais representações do recurso de destino pela carga de dados da requisição.

DELETE

O método `DELETE` remove um recurso específico.

Verbos HTTP

Principais

GET

O método GET solicita equisições utilizando o método GET devem reto

POST

O método POST é u m recurso específico, frequentemente cau u efeitos colaterais no servidor.

PUT

O método PUT substi e destino pela carga de dados da requisição.

DELETE

O método DELETE remove um recurso específico.

Para mais métodos de requisição

https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods

Status HTTP

Os códigos de status das respostas HTTP indicam se uma requisição HTTP foi corretamente concluída. As respostas são agrupadas em cinco classes:

- Respostas de informação (100-199),
- Respostas de sucesso (200-299),
- Redirecionamentos (300-399)
- Erros do cliente (400-499)
- Erros do servidor (500-599).

Status HTTP

Principais

200 OK: Estas requisição foi bem sucedida. O significado do sucesso varia de acordo com o método HTTP

201 Created: A requisição foi bem sucedida e um novo recurso foi criado como resultado. Esta é uma tipica resposta enviada após uma requisição POST.

400 Bad Request: Essa resposta significa que o servidor não entendeu a requisição pois está com uma sintaxe inválida.

401 Unauthorized: Embora o padrão HTTP especifique "unauthorized", semanticamente, essa resposta significa "unauthenticated". Ou seja, o cliente deve se autenticar para obter a resposta solicitada.

404 Not Found: O servidor não pode encontrar o recurso solicitado. Este código de resposta talvez seja o mais famoso devido à frequência com que acontece na web.

500 Internal Server Error: O servidor encontrou uma situação com a qual não sabe lidar.

Status HTTP

Principais

200 OK: Estas requisição foi bem sucedida. O significado do sucesso varia de acordo com o método HTTP

201 Created: A requisiç rso foi criado como resultado. Esta é uma tip ão POST.

400 Bad Request: Essa r tendeu a requisição pois está com uma sintax

401 Unauthorized: E e "unauthorized", semanticamente, essa re seja, o cliente deve se autenticar para obter

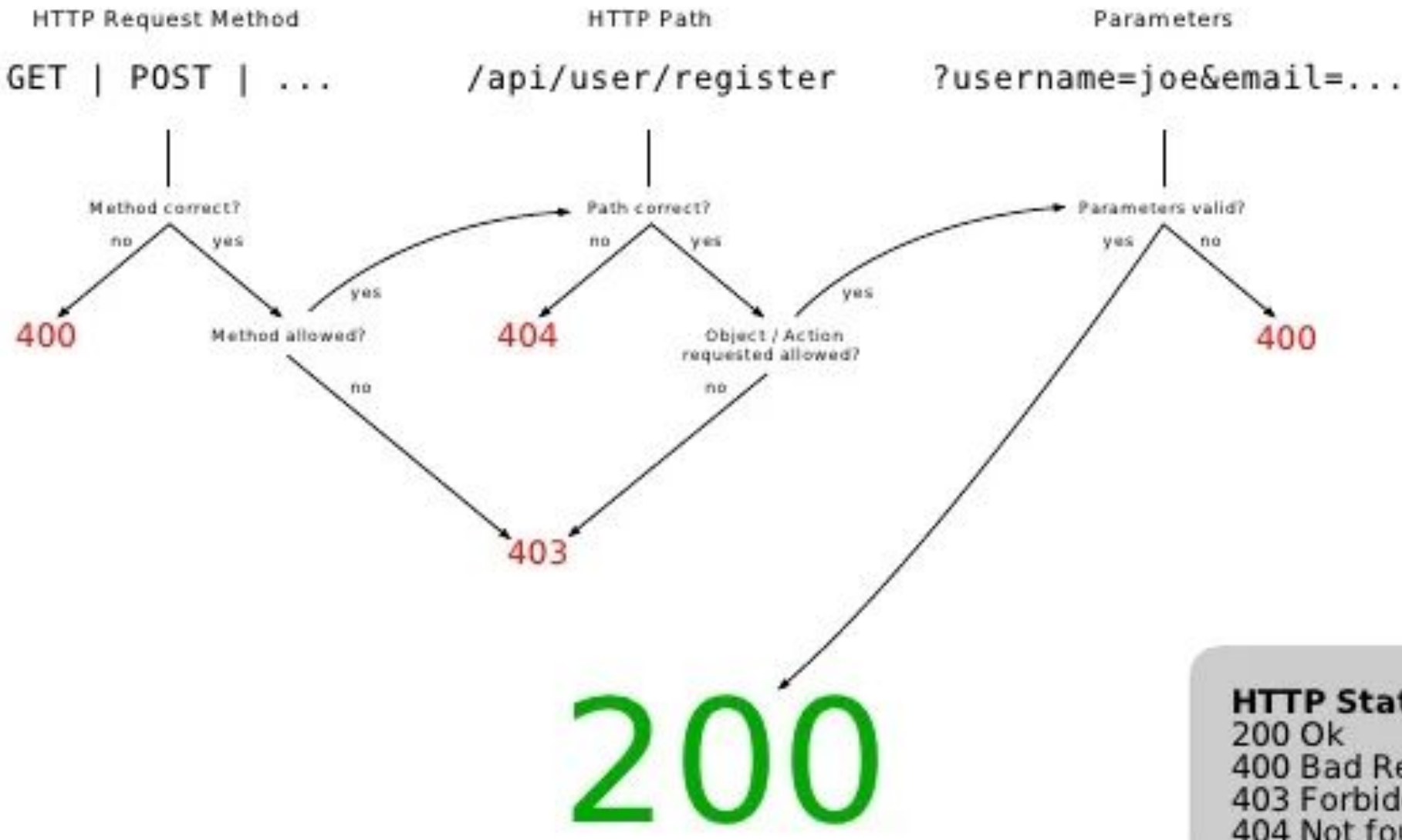
404 Not Found: O servidor não pode encontrar o recurso solicitado. Este código de resposta talvez seja o mais famoso devido à frequência com que acontece na web.

500 Internal Server Error: O servidor encontrou uma situação com a qual não sabe lidar.

Para mais status

https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status

REST HTTP Response Code Cheat Sheet



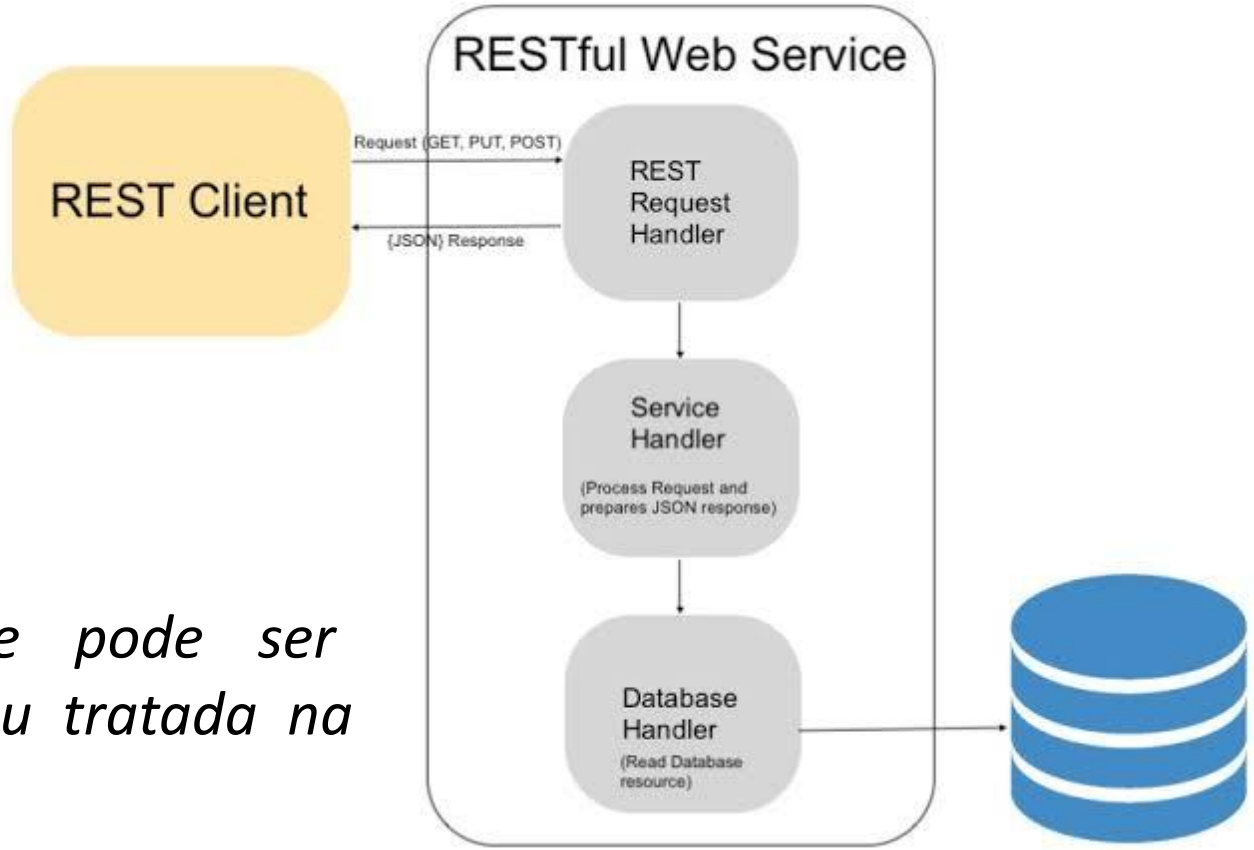
HTTP Status Codes
200 Ok
400 Bad Request
403 Forbidden
404 Not found

Arquitetura Restful

RESTful Applications:

Aplicações que utilizam o padrão REST para comunicação. Em uma aplicação RESTful a comunicação é realizada preferencialmente via URLs (representa um *resource*).

`http://myserver.com/catalog/item/1729`



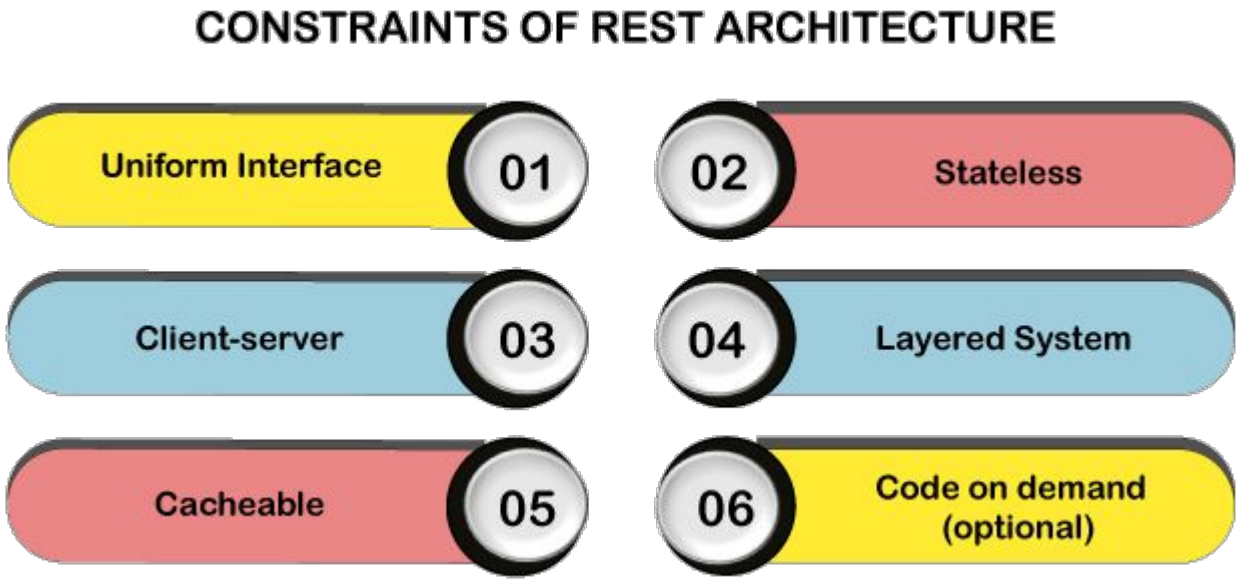
*Um **recurso** é uma entidade que pode ser identificada, nomeada, endereçada ou tratada na web.*

Arquitetura Restful

O que define uma aplicação Restful?

O REST é o conjunto de restrições abaixo para organização de APIs Web.

- **Cliente–Servidor**
- **Sem estado**
- **Cacheable**
- **Interface uniforme**
- **Sistema em Camada**
- **Código sob demanda (optional)**



Arquitetura Restful

Cliente-servidor - separando as questões da interface do usuário das questões de armazenamento de dados, melhoramos a portabilidade da interface do usuário em várias plataformas e melhoramos a escalabilidade simplificando os componentes do servidor (Ex: front-end em React e back-end em Spring).

Sem estado - cada solicitação do cliente para o servidor deve conter todas as informações necessárias para entender a solicitação e não pode tirar proveito de qualquer contexto armazenado no servidor. O estado da sessão é, portanto, mantido inteiramente no cliente (Ex: Na memória do servidor não há nenhum dado do cliente mantido, apenas no momento de atender a uma requisição).

Arquitetura Restful

Armazenável em cache - as restrições de cache exigem que os dados em uma resposta a uma solicitação sejam implicitamente ou explicitamente rotulados como armazenáveis ou não armazenáveis em cache (do cliente). Se uma resposta puder ser armazenada em cache, o cache do cliente terá o direito de reutilizar os dados de resposta para solicitações equivalentes posteriores.

Interface uniforme - ao aplicar o princípio geral da engenharia de software à interface do componente (webservice), a arquitetura geral do sistema é simplificada e a visibilidade das interações é aprimorada. Para obter uma interface uniforme, várias restrições arquitetônicas são necessárias para orientar o comportamento dos componentes. REST é definido por quatro restrições de interface: **identificação de recursos**; **manipulação de recursos por meio de representações**; **mensagens autodescritivas**; e **hipermídia como motor de estado do aplicativo**.

Arquitetura Restful

Sistema em camadas - O estilo de sistema em camadas permite que uma arquitetura seja composta de camadas hierárquicas, restringindo o comportamento do componente de forma que cada componente não possa "ver" além da camada imediata com a qual está interagindo (Ex: camada de aplicação não se comunica com a camada de dados diretamente).

Código sob demanda (opcional) - O REST permite que a funcionalidade do cliente seja estendida ao baixar e executar o código na forma de miniaplicativos ou scripts. Isso simplifica os clientes, reduzindo o número de recursos necessários para serem pré-implementados.

Arquitetura Restful

Siste que
uma do o
com não
poss
Códi lade
do c de
mini o o
núm

Se a arquitetura de uma aplicação não segue esses princípios, então ela não é Restful. Veja mais em:

https://restfulapi.net/

Veja uma implementação em:
https://www.youtube.com/watch?v=ghTrp1x_1As

Arquitetura Restful

Para nós, o que é importante?

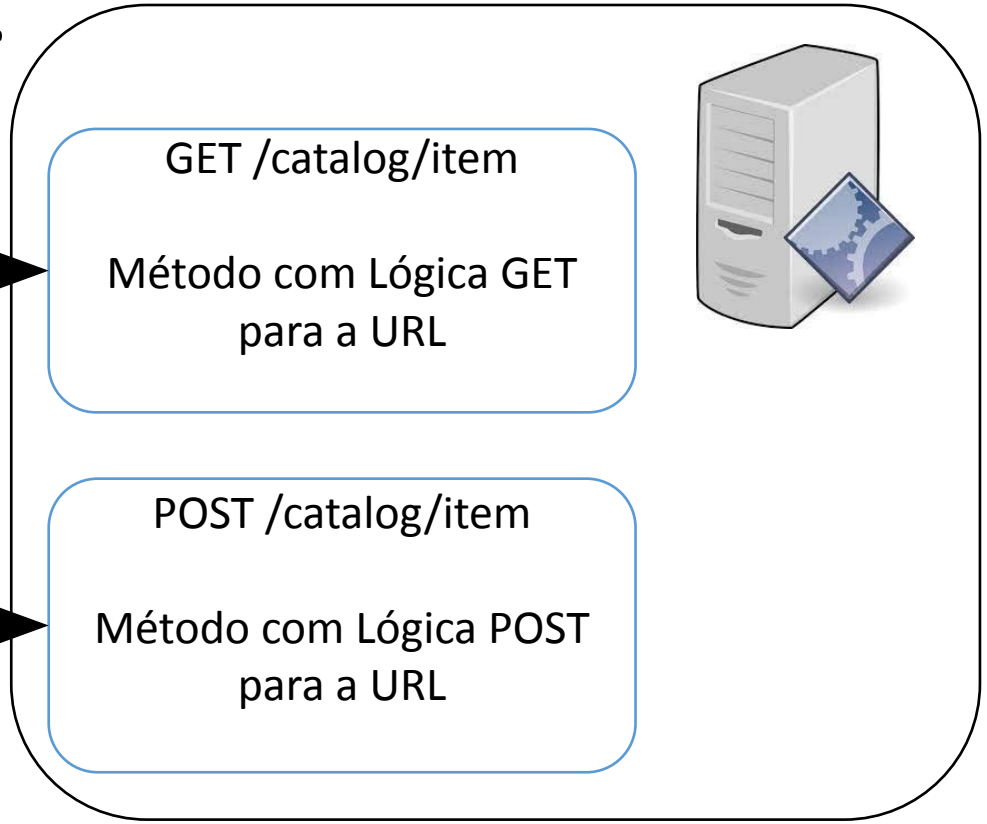
- Aplicação Cliente/Servidor se comunicando via HTTP;
- Servidor não deve armazenar nenhuma informação do Cliente em sessões;
- Ter uma interface uniforme (Verbos HTTP + *Paths* bem definidos) entre os componentes para que as informações sejam transferidas em um formato padronizado (vamos utilizar o JSON).

Como definir uma API REST?

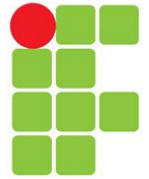
Os verbos HTTP mais importantes para criar uma API RESTful são GET, POST, PUT e DELETE. **A combinação dos verbos com as URLs dos recursos formam identificadores únicos.**

GET http://myserver.com/catalog/item

POST http://myserver.com/catalog/item



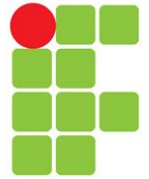
Como definir uma API REST?



Aqui estão algumas regras gerais que as APIs REST seguem:

- Métodos HTTP como GET, POST, UPDATE e DELETE informam o servidor sobre a ação a ser executada, usualmente relacionadas a operações CRUD.
- Método HTTP diferente invocado para uma mesma URL fornece funcionalidade diferente. (**`GET /user`** é diferente de **`POST /user`**)
- Os códigos de status de resposta HTTP padrão são retornados pelo servidor indicando sucesso ou falha.
- APIs REST podem retornar respostas JSON ou XML.

Como definir uma API REST?



Aqui estão algumas regras gerais que as APIs REST seguem:

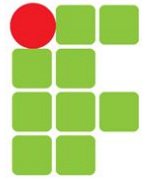
- Os recursos fazem parte de URLs, ex: ***`/users`***, ***`/perfil`***, ***`/registro`***.
- Para cada recurso, duas URLs são geralmente implementadas:
  - uma para a coleção, como **`/users`**,
  - e uma para um elemento específico, como **`/users/123`**.
- Substantivos são usados em vez de verbos para recursos.
  - Exemplo: em vez de **`/getUserInfo/U123`**, use **`/users/U123`**.

Como definir uma API REST?

Exemplo: Operações CRUD, Verbos HTTP e convenções REST

Operation	HTTP verb	URL: /users	URL: /users/U123
Create	POST	Create a new user	Not applicable
Read	GET	List all users	Retrieve user U123
Update	PUT or PATCH	Batch update users	Update user U123
Delete	DELETE	Delete all users	Delete user U123

Como definir uma API REST?



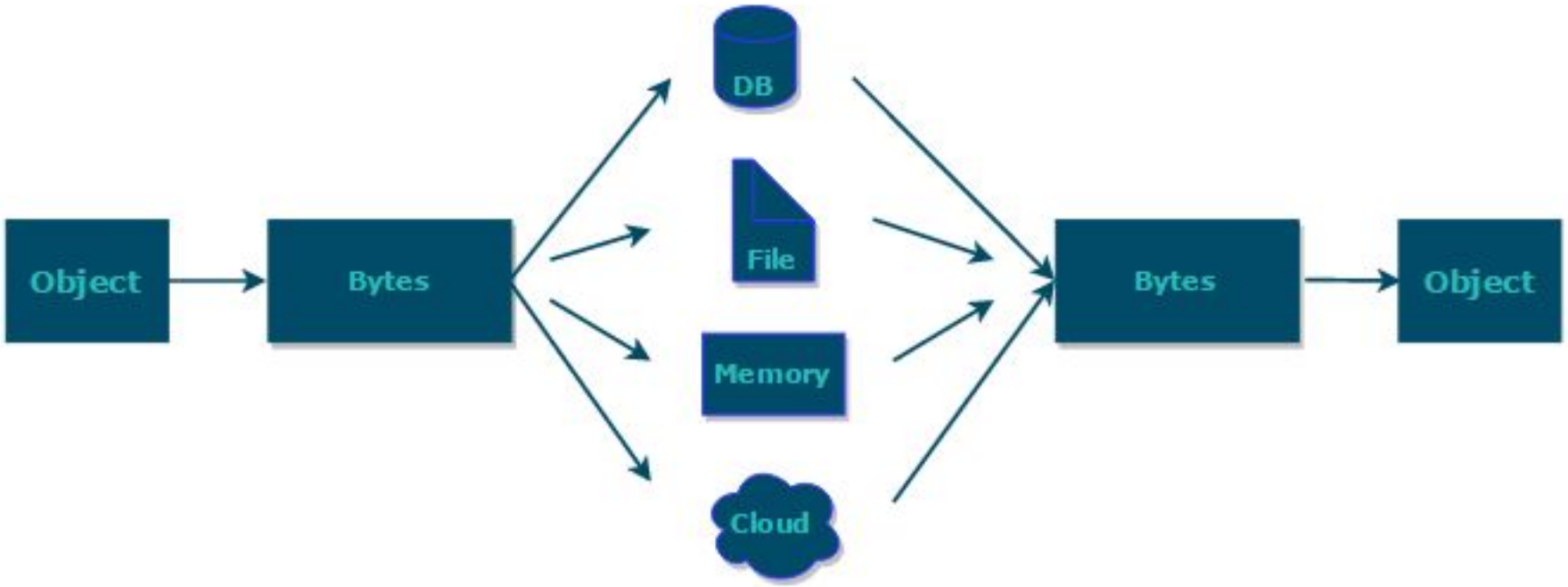
Exemplo: Operações CRUD, Verbos HTTP e convenções REST

Endpoint	Método	Ação
/users	GET	Retorna a lista de usuários
/users	POST	Insere um novo usuário
/users/{id}	GET	Retorna o usuário com id = {id}
/users/{id}	PUT	Substitui os dados do usuário com id = {id}
/users/{id}	PATCH	Altera items dos dados do usuário com id = {id}
/users/{id}	DELETE	Remove o usuário com id = {id}

Fonte: https://blog.mbeck.com.br/api-rest-e-os-verbos-http-46e189085e21

Serialização de objetos

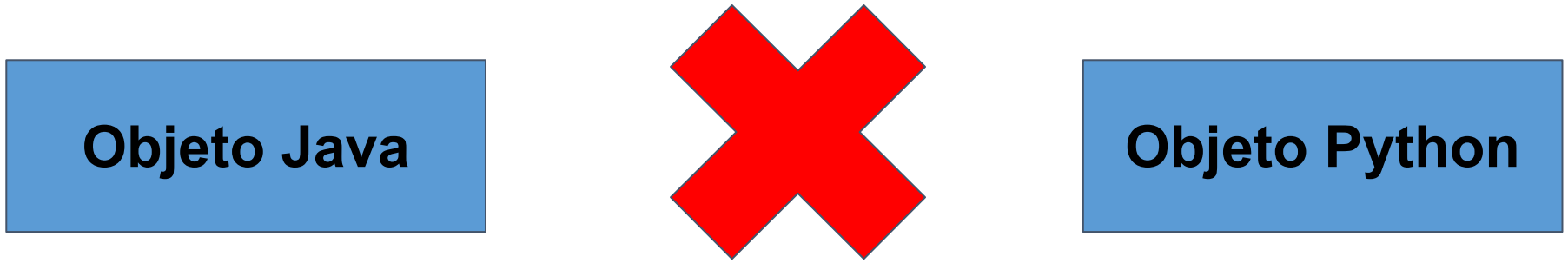
É um processo para converter uma estrutura de dados ou um objeto em um formato que possa ser armazenado ou transferido.



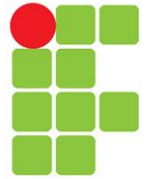
Serialização de objetos

A serialização possibilitará capturar o estado do objeto ou a estrutura de dados, e transformar em uma cadeia de bytes quando for necessário. Torna-se possível também, recuperar os bytes persistidos para fazer o processo inverso, e ter os dados de volta para a aplicação em execução. Isso é necessário especialmente quando precisamos transferir objetos de uma linguagem para outra através de um formato comum.

Ex:



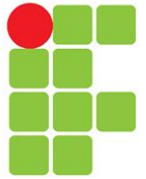
Serialização de objetos



Quais formatos de serialização?

- Stream de bytes, byte-stream-based encoding
- XML
- **JSON**
- YAML

Serialização de objetos



JSON

- Este acrônimo significa “JavaScript Object Notation” e, como o nome implica, o formato usa objeto JavaScript e sintaxe literal de array para converter dados estruturas que consistem em objetos e matrizes em strings.
- JSON suporta números primitivo, strings e também os valores true, false e null, bem como matrizes e objetos construídos a partir desses valores primitivos.
- JavaScript suporta serialização e desserialização JSON com as duas funções **JSON.stringify()** e **JSON.parse()**.
- No Java a serialização é realizada de forma automática pelo Spring.

Serialização de objetos

Exemplo: Representando dados de um empresa

```
{
  "company": "mycompany",
  "companycontacts": {
    "phone": "123-123-1234",
    "email": "myemail@domain.com"
  },
  "employees": [
    {
      "id": 101,
      "name": "John",
      "contacts": [
        "email1@employee1.com",
        "email2@employee1.com"
      ]
    },
    {
      "id": 102,
      "name": "William",
      "contacts": null
    }
  ]
}
```

JSON Object

String Value

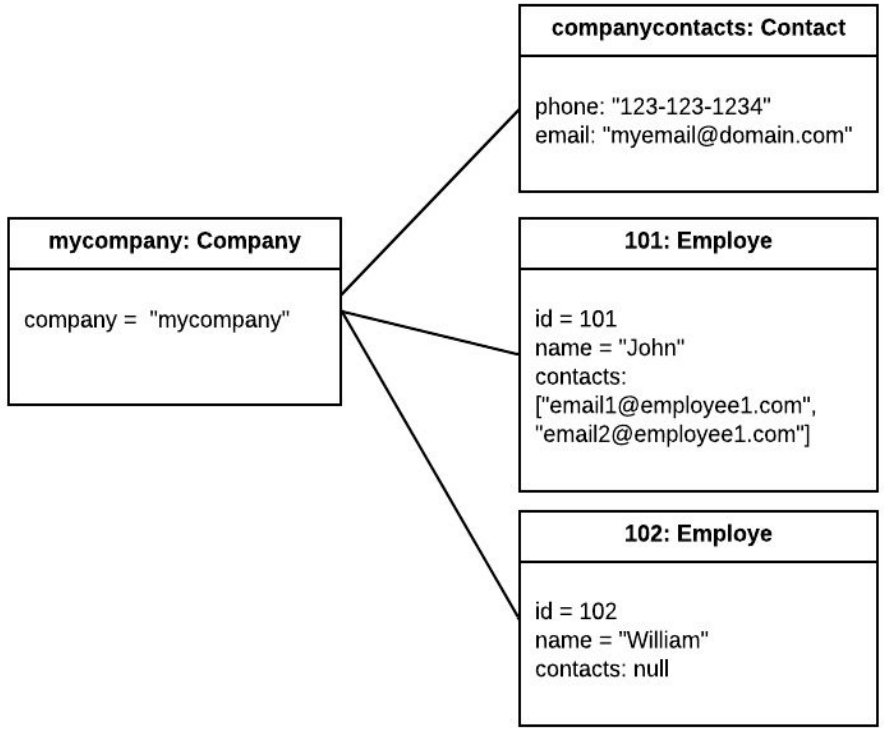
Object Inside Object

JSON Array

Array Inside Array

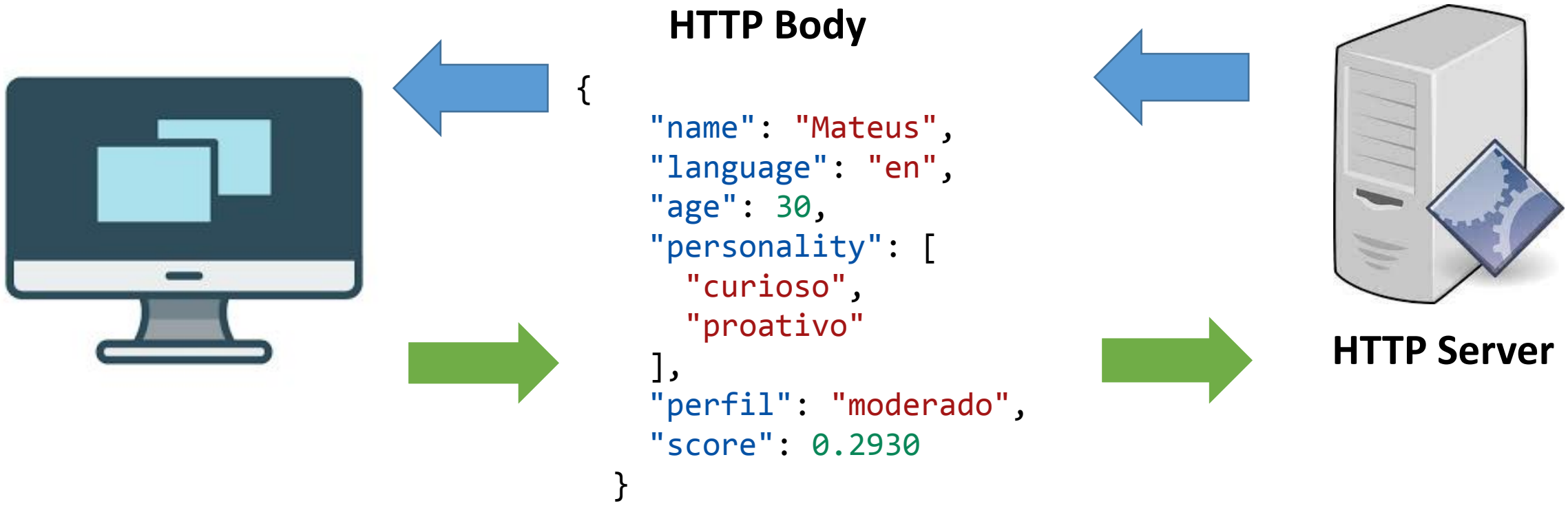
Number Value

Null Value



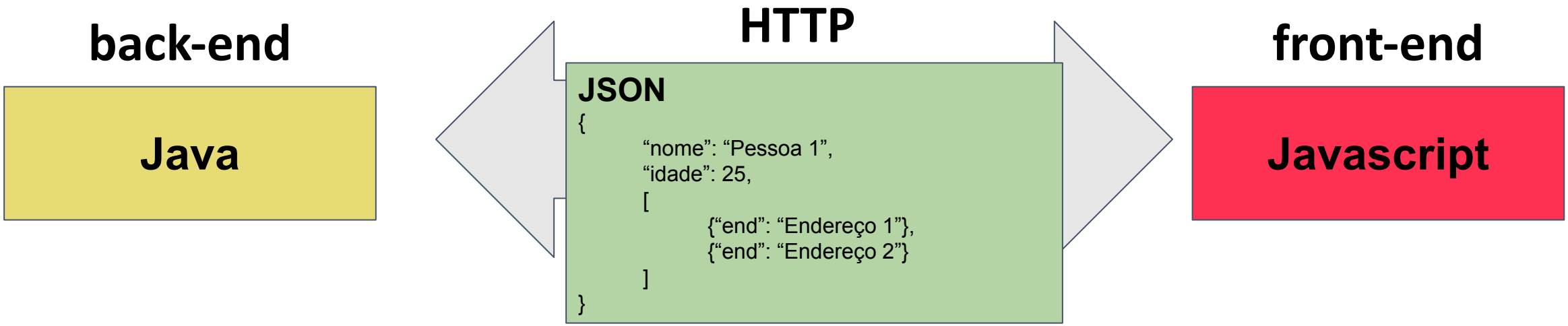
Serialização de objetos

REST usa várias representações para representar um recurso como texto, JSON, XML. JSON (Javascript Object Notation) é o mais popular.

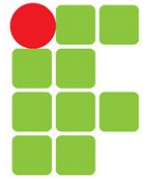


Serialização de objetos

REST usa várias representações para representar um recurso como texto, JSON, XML. JSON (Javascript Object Notation) é o mais popular.



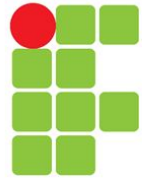
Criando WebServices com Spring MVC



Spring Web MVC é o framework web original construído na API Servlet (é uma interface de programação de aplicativos (API) que permite a criação de aplicações web dinâmicas no Java) e foi incluído no Spring Framework desde o início.

O nome formal, **"Spring Web MVC"**, vem do nome do seu módulo de origem (spring-webmvc), mas é mais comumente conhecido como "Spring MVC".

Criando WebServices com Spring MVC



Ele promove a modularidade, a reutilização de código e a separação de preocupações, permitindo que os desenvolvedores construam aplicativos web robustos e escaláveis.

Alguns recursos ofertados pelo Spring
https://docs.spring.io/spring-framework/reference/web/webmvc.html

Filters
HTTP Message Conversion
Annotated Controllers
Functional Endpoints
URI Links

Asynchronous Requests
CORS
Error Responses
Web Security
HTTP Caching

Criando WebServices com Spring MVC

Os recursos do Spring MVC são acessados principalmente por meio de anotações (annotation)

MANIPULAÇÃO DE REQUISIÇÕES

- **@RestController**
  - Indica que a classe é um controlador REST.
- **@RequestMapping**
  - Define a URL base do controlador e o método HTTP.
- **@GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping**
  - Mapeiam métodos HTTP específicos para handlers.

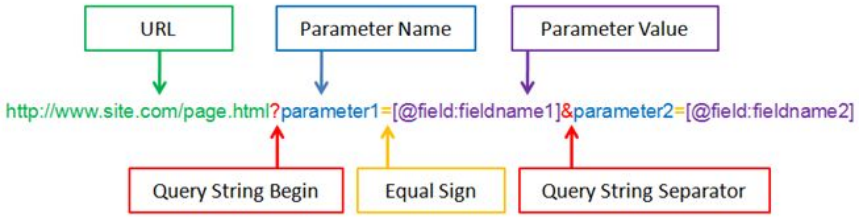


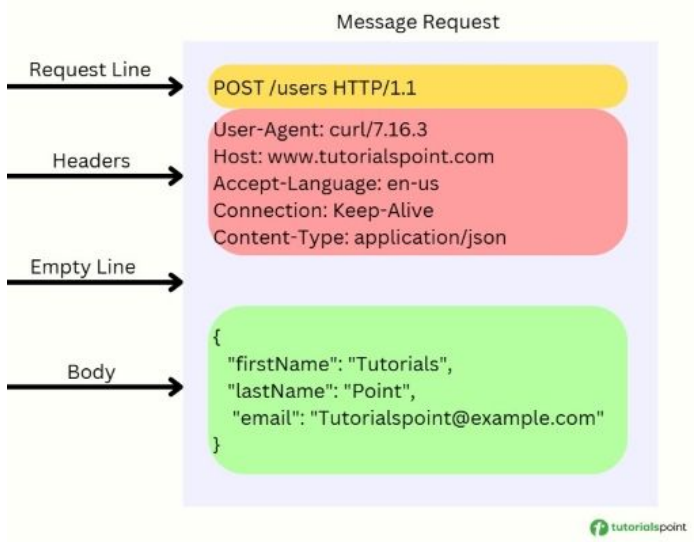
Criando WebServices com Spring MVC

Os recursos do Spring MVC são acessados principalmente por meio de anotações (annotation)

OBTENÇÃO DE DADOS DA REQUISIÇÃO

- **@RequestParam**
  - Captura parâmetros da query string (?chave=valor).
- **@PathVariable**
  - Captura valores da URL (/recurso/{id}).
- **@RequestBody**
  - Obtém o corpo da requisição, útil para POST e PUT.
- **@RequestHeader**
  - Obtém valores dos cabeçalhos HTTP.





Criando WebServices com Spring MVC

Os recursos do Spring MVC são acessados principalmente por meio de anotações (annotation)

MANIPULAÇÃO DA RESPOSTA

- **@ResponseEntity<T>**
  - Classe que oferece controle sobre a resposta HTTP, definindo código de status, header e body.

@ResponseStatus

- Define o status HTTP diretamente no método.



Criando WebServices com Spring MVC

Os recursos do Spring MVC são acessados principalmente por meio de anotações (annotation)

CONFIGURAÇÃO DE CORS

- **@CrossOrigin**
  - Permite configurações de CORS (https://aws.amazon.com/pt/what-is/cross-origin-resource-sharing/) diretamente nos controladores.

FILTROS E INTERCEPTADORES

- **HandlerInterceptor**
  - Intercepta requisições antes e depois de chegarem à classe controladora dentro do Spring. Utilizado em contexto de regra de negócio (Ex: Só usuários Premium podem acessar um dado recurso).
- **Filters**
  - interceptam as requisições antes que cheguem ao Controller do Spring e as respostas antes de saírem para o cliente. São ideais para tarefas transversais como log, autenticação e compressão de dados.



Associando o HTTP com recursos do Spring

Requisição HTTP

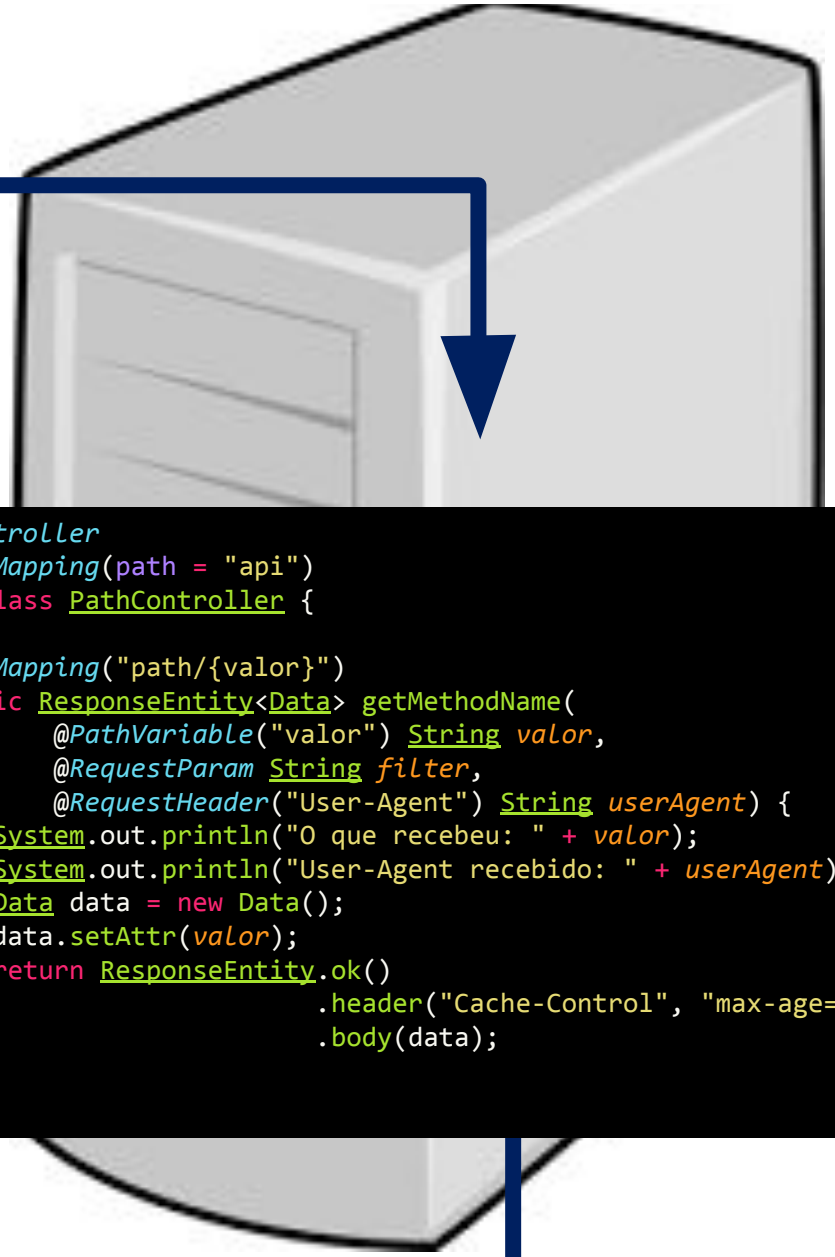


Client

```
> GET /api/path/meudado?filter=valor HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.88.1
> Accept: */*
```

Requisição HTTP

Server com Spring Web MVC



```java
@RestController
@RequestMapping(path = "api")
public class PathController {

    @GetMapping("path/{valor}")
    public ResponseEntity<Data> getMethodName(
            @PathVariable("valor") String valor,
            @RequestParam String filter,
            @RequestHeader("User-Agent") String userAgent) {
        System.out.println("O que recebeu: " + valor);
        System.out.println("User-Agent recebido: " + userAgent);
        Data data = new Data();
        data.setAttr(valor);
        return ResponseEntity.ok()
                             .header("Cache-Control", "max-age=3600")
                             .body(data);
    }
}
```

Resposta HTTP



Client

```
< HTTP/1.1 200
< Cache-Control: max-age=3600
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Wed, 26 Mar 2025 14:16:14 GMT
<
{"attr":"meudado"}
```

Resposta HTTP

Associando o HTTP com recursos do Spring

Server com Spring Web MVC

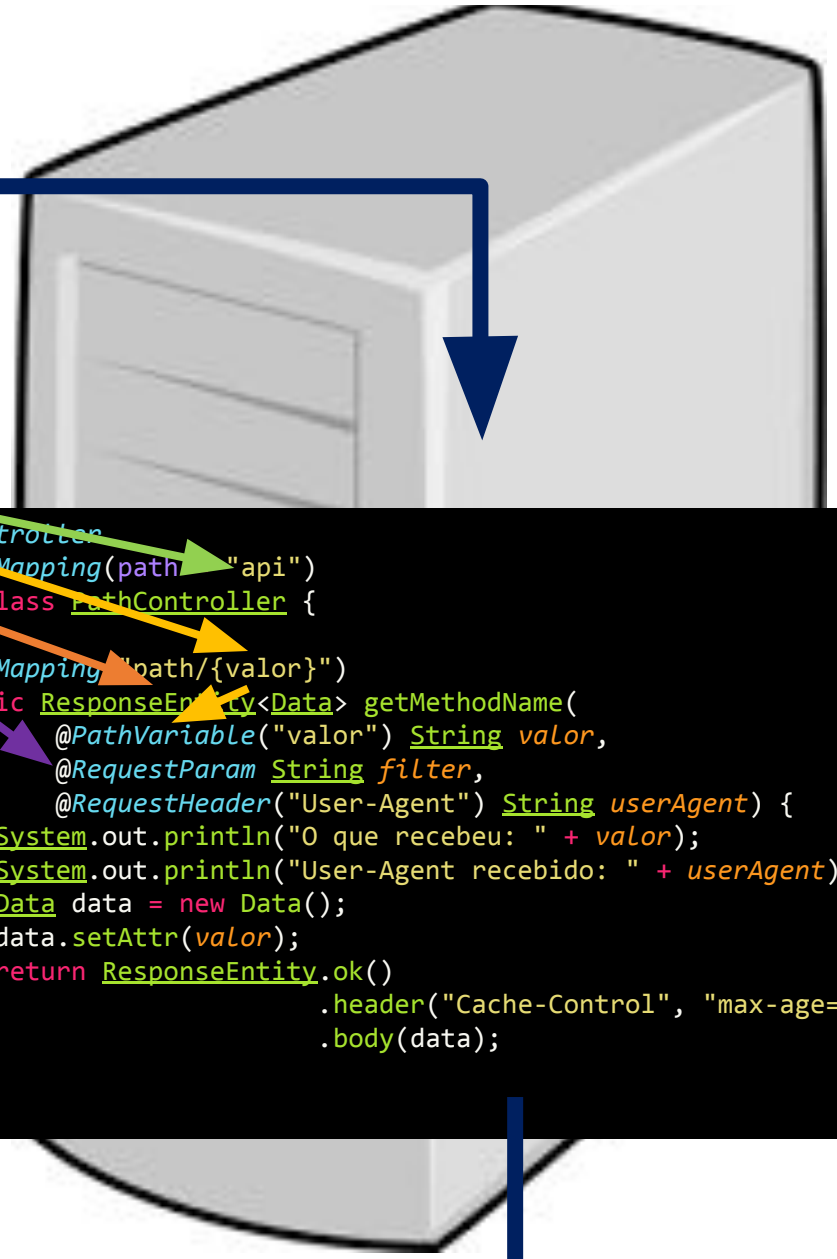
Requisição HTTP



Client

```
> GET /api/path/meudado?filter=valor HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.88.1
> Accept: */*
```

Requisição HTTP



```
@RestController
@RequestMapping(path = "api")
public class PathController {

    @GetMapping("path/{valor}")
    public ResponseEntity<Data> getMethodName(
            @PathVariable("valor") String valor,
            @RequestParam String filter,
            @RequestHeader("User-Agent") String userAgent) {
        System.out.println("O que recebeu: " + valor);
        System.out.println("User-Agent recebido: " + userAgent);
        Data data = new Data();
        data.setAttr(valor);
        return ResponseEntity.ok()
                             .header("Cache-Control", "max-age=3600")
                             .body(data);
    }
}
```

Resposta HTTP



Client

```
< HTTP/1.1 200
< Cache-Control: max-age=3600
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Wed, 26 Mar 2025 14:16:14 GMT
<
{"attr":"meudado"}
```

Resposta HTTP

Associando o HTTP com recursos do Spring

Server com Spring Web MVC

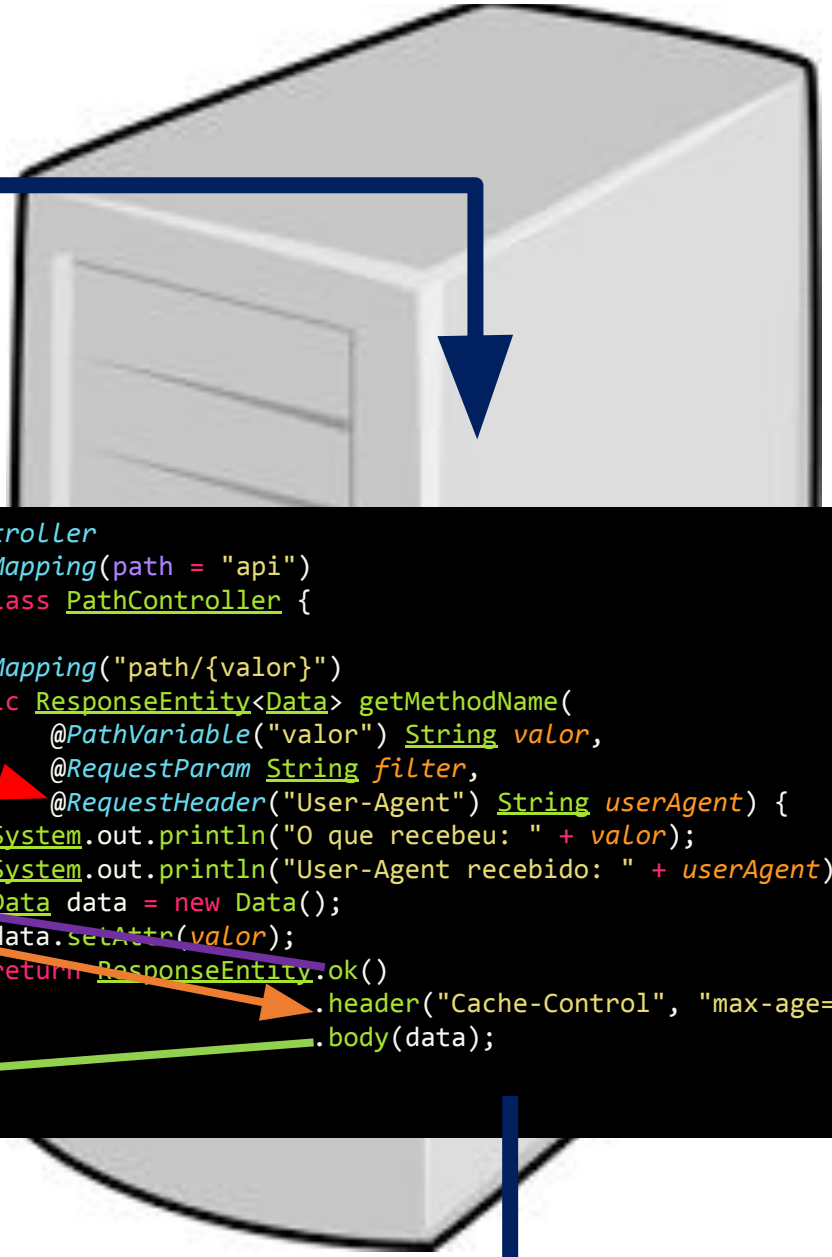
Requisição HTTP



Client

Requisição HTTP

```
> GET /api/path/meudado?filter=valor HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.88.1
> Accept: */*
```



```java
@RestController
@RequestMapping(path = "api")
public class PathController {

    @GetMapping("path/{valor}")
    public ResponseEntity<Data> getMethodName(
            @PathVariable("valor") String valor,
            @RequestParam String filter,
            @RequestHeader("User-Agent") String userAgent) {
        System.out.println("O que recebeu: " + valor);
        System.out.println("User-Agent recebido: " + userAgent);
        Data data = new Data();
        data.setAttr(valor);
        return ResponseEntity.ok()
                .header("Cache-Control", "max-age=3600")
                .body(data);
    }
}
```

Resposta HTTP

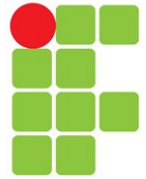
```
< HTTP/1.1 200
< Cache-Control: max-age=3600
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Wed, 26 Mar 2025 14:16:14 GMT
<
{"attr":"meudado"}
```



Client

Resposta HTTP

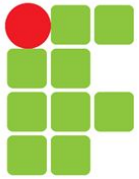
Criando WebServices com Spring MVC



Vamos fazer um exemplo simples envolvendo as anotações abordadas.

1. Inicialmente crie um projeto Spring, como orientado na aula anterior.
2. Vamos criar uma classe chamada **`User`**, com login e senha, para realizar os testes necessários.
3. Vamos criar a classe **`UserController`** para servir de nosso WebService REST.

Para os testes vamos utilizar essa classe User



```java
public class User {
    private String login;
    private String password;

    public String getLogin(){
        return login;
    }
    public void setLogin(String login){
        this.login = login;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

Exemplos básicos com o Spring MVC



```java
@RestController
public class UserController {
        @GetMapping("/user")
    public ResponseEntity<String> endPoint1(){
        return ResponseEntity.ok("Exemplo API Rest");
    }

    @GetMapping("/user/{login}")
    public ResponseEntity<String> endPoint2(@PathVariable("login") Long valor){
        return ResponseEntity.ok("Parâmetros recebido LOGIN: "  + valor);
    }

    @PostMapping("/user")
    public ResponseEntity<User> endPoint3(@RequestBody User novoObjeto){
        return ResponseEntity.created(
            URI.create("user/" + novoObjeto.getLogin() ) // HEADER location: user/ifsp
        ).body(novoObjeto);
    }
  /* Continua */
```

Exemplos básicos com o Spring MVC



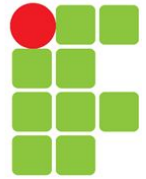
```
    @PutMapping("/user")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public ResponseEntity<User> endPoint4(@RequestBody User objetoParaAtualizar){
        return ResponseEntity.ok(objetoParaAtualizar);
    }

    @DeleteMapping("/user/{login}")
    public ResponseEntity<String> endPoint5(@PathVariable("login") String login){
        if(login.equals("user1"))
            return ResponseEntity.ok("Remoção de informação com LOGIN " + login + "realizada");
        else{
            return ResponseEntity.ok("Objeto para id " + login + "não encontrado");
        }
    }
}
```

Atenção:

- **@ResponseStatus** serve para definir o status HTTP padrão em caso de sucesso na operação.
- Por padrão o status é 200, que corresponde a **@ResponseStatus(HttpStatus.OK)**

Criando WebServices com Spring MVC



Acessando nossa API (Interface Uniforme)

No navegador, testes as URLs com verbo GET.

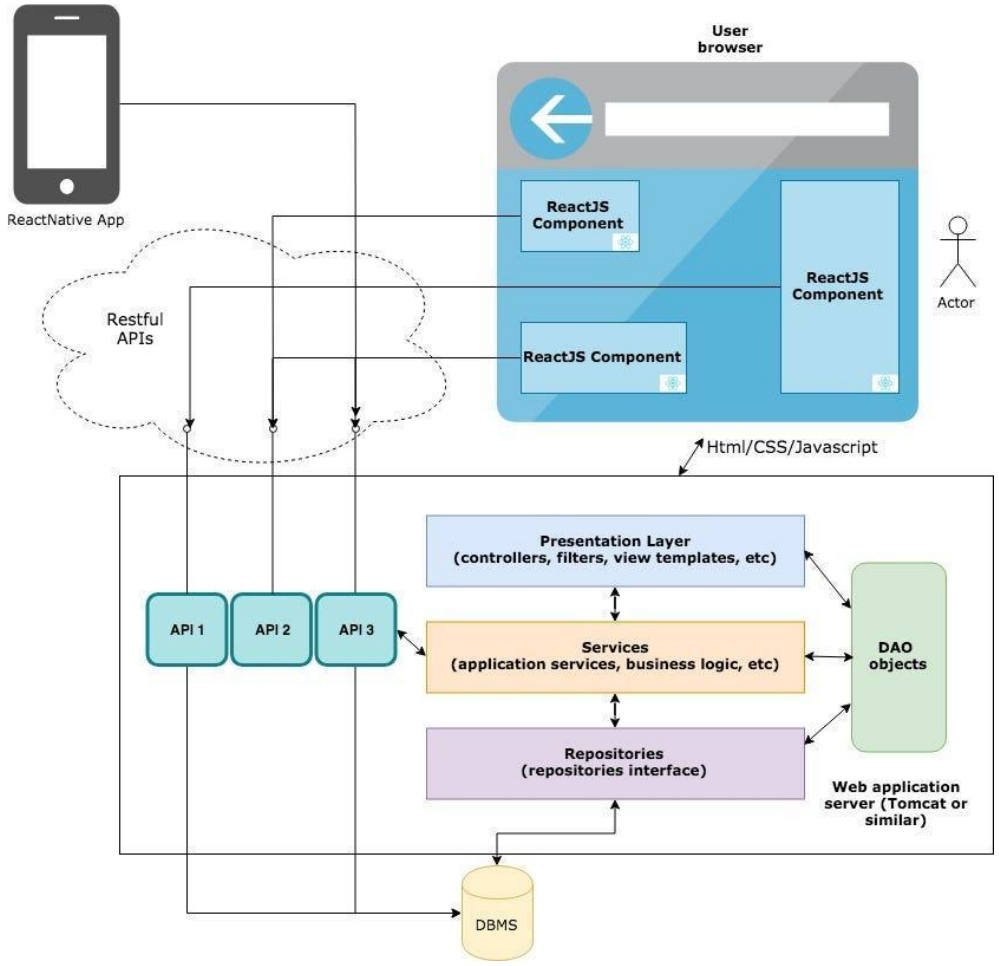
Verbo	A URL mapeia para...	... Este Endpoint
GET	localhost:8080/user	@GetMapping("/user")
GET	localhost:8080/user/99	@GetMapping("/user/{login}")
POST	localhost:8080/user	*@PostMapping*("/user")
PUT	localhost:8080/user	*@PutMapping*("/user")
DELETE	localhost:8080/user/1	@DeleteMapping("/user/{login}")

Para o POST, PUT (onde usamos **@RequestBody**) e DELETE**,** precisamos de ferramentas específicas.

Criando WebServices com Spring MVC

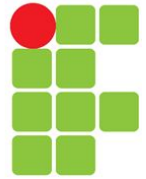
Algumas ferramentas para teste de API

- A arquitetura da figura ilustra como a aplicação desenvolvida será organizada.
- Entretanto, ainda não temos uma interface gráfica para “conectar” a uma API Web que criarmos.
- Podemos utiliza ferramentas como Clientes enquanto não temos as interfaces web disponíveis.



Fonte: https://medium.com/swlh/moving-from-java-spring-framework-to-a-reactjs-apis-architecture-dd85787cc616

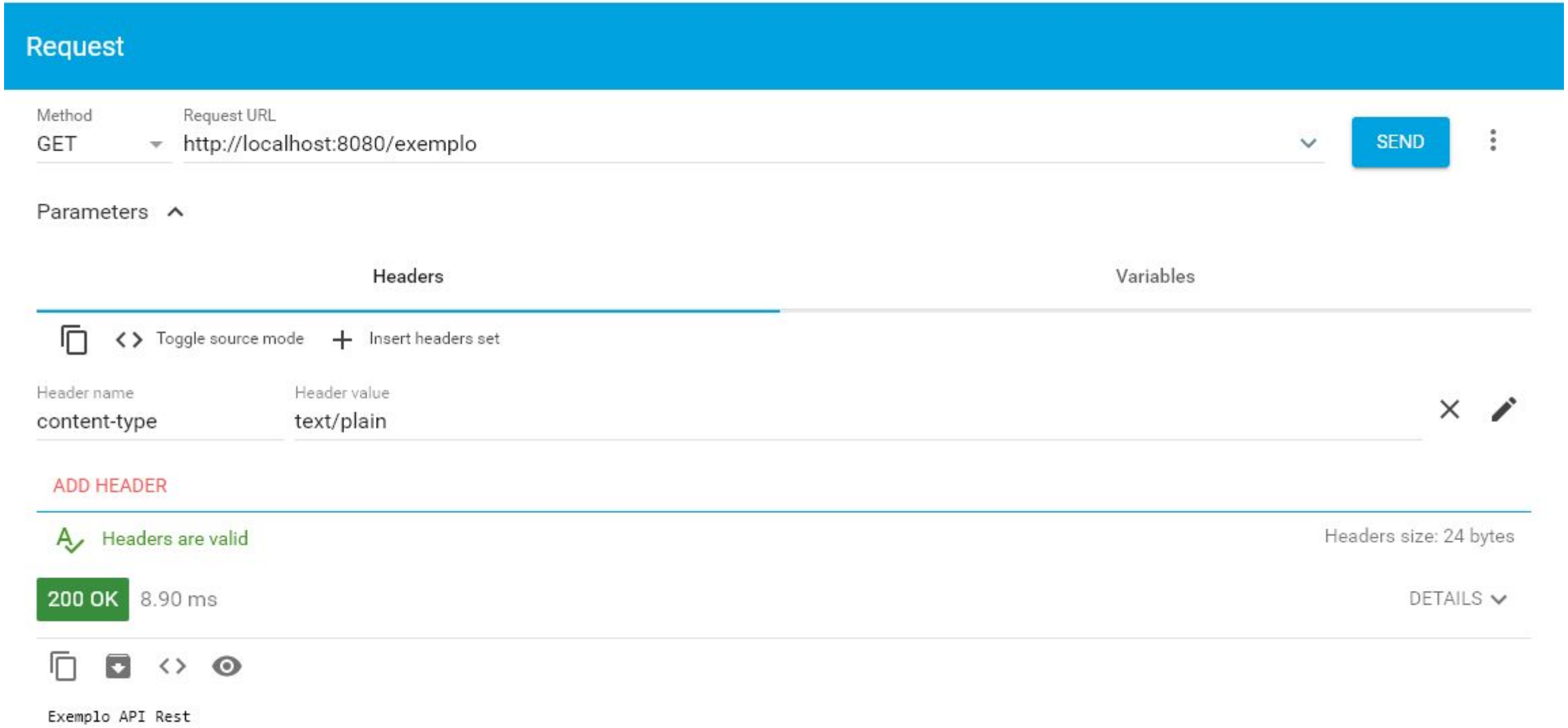
Criando WebServices com Spring MVC



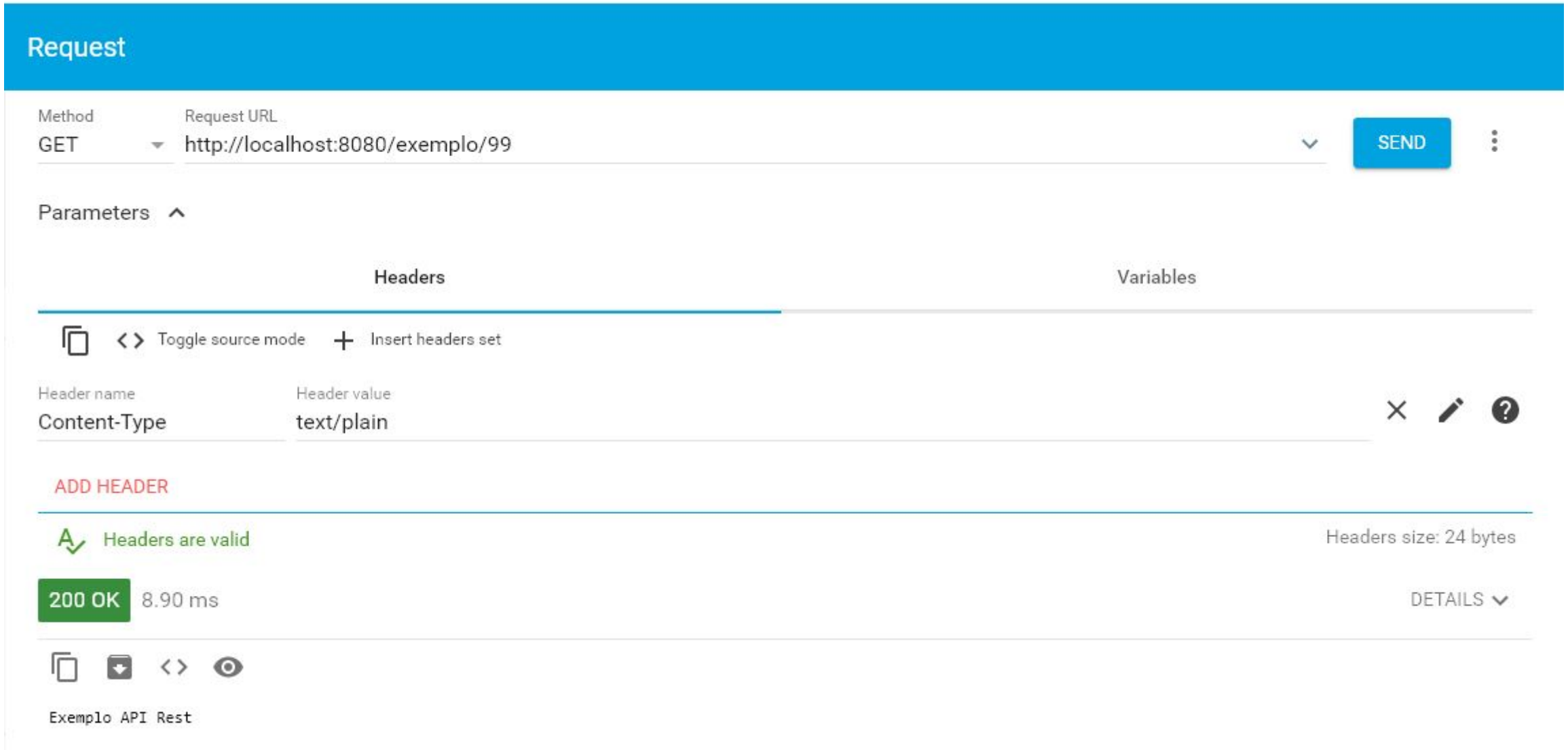
Algumas ferramentas para teste de API

- **Postman**
- **Thunder Client (VSCode)**
- SoapUI
- Assertible
- Insomnia
- **Advanced REST cliente (Chrome Plugin)**
- RESTer (Firefox Plugin)

Configuração GET



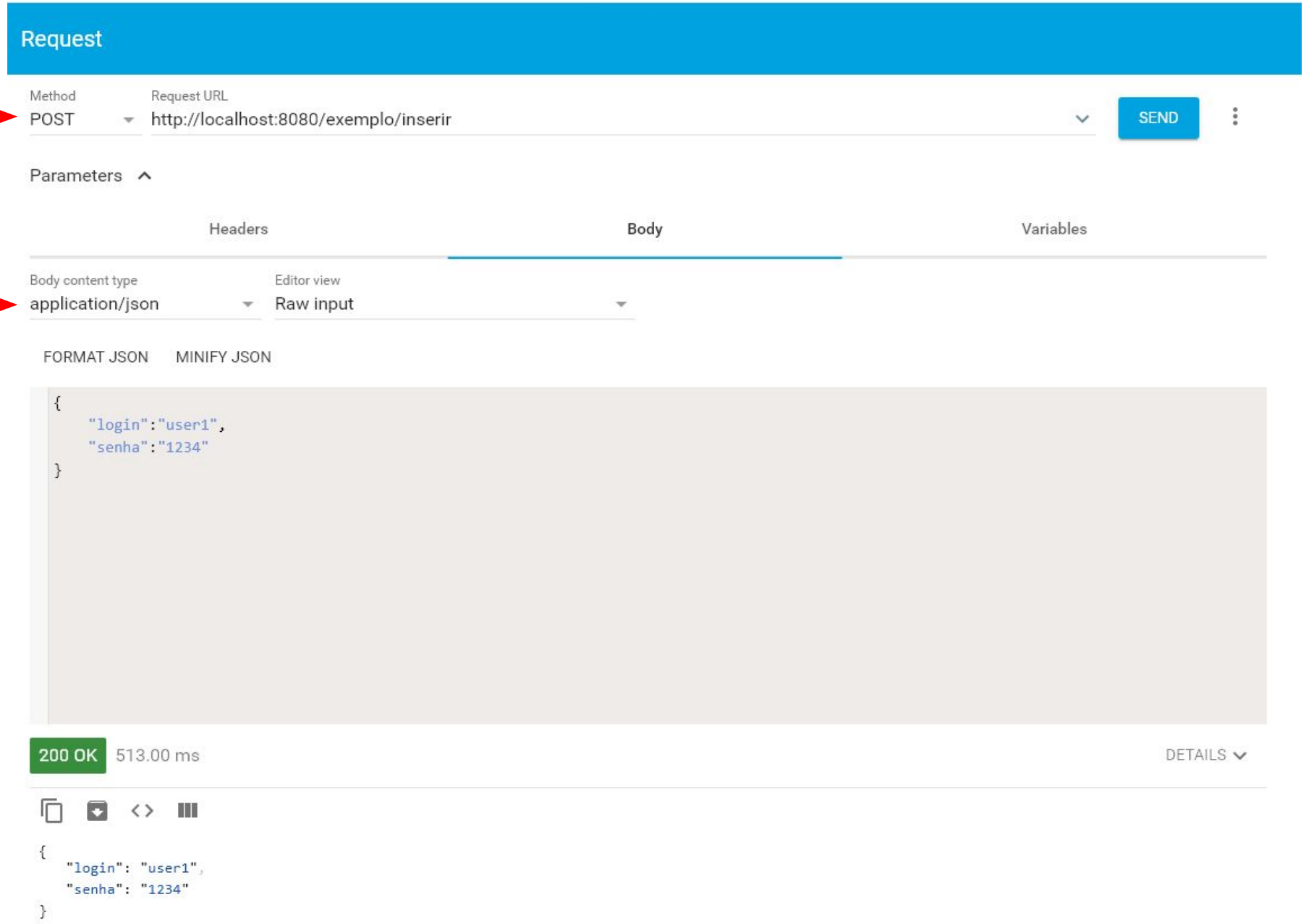
Configuração GET (Com parâmetros)



Configuração POST

Atenção com a mudança do método HTTP.

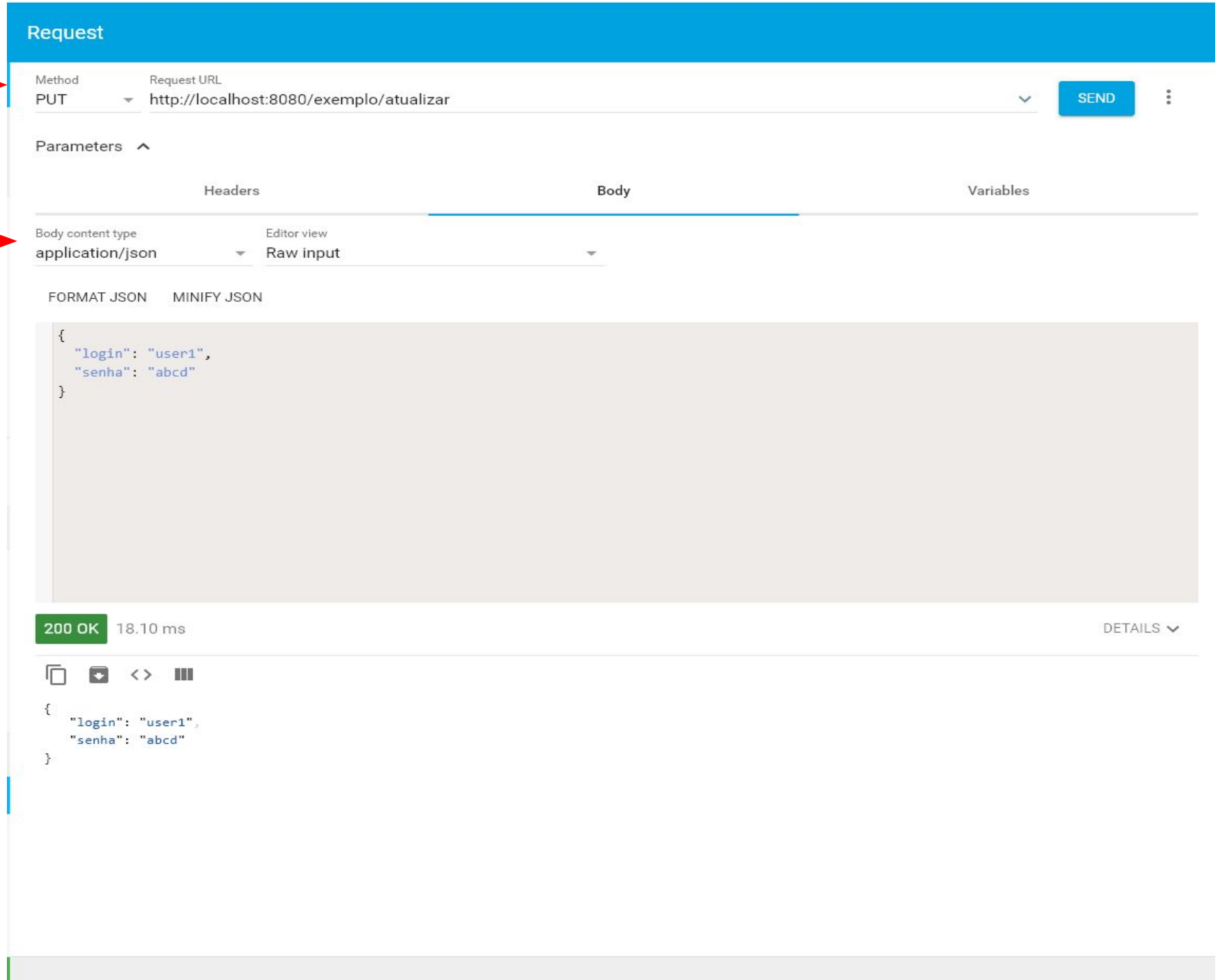
Atenção com o formato do dado a ser transmitido. No caso estamos usando JSON.



Configuração PUT

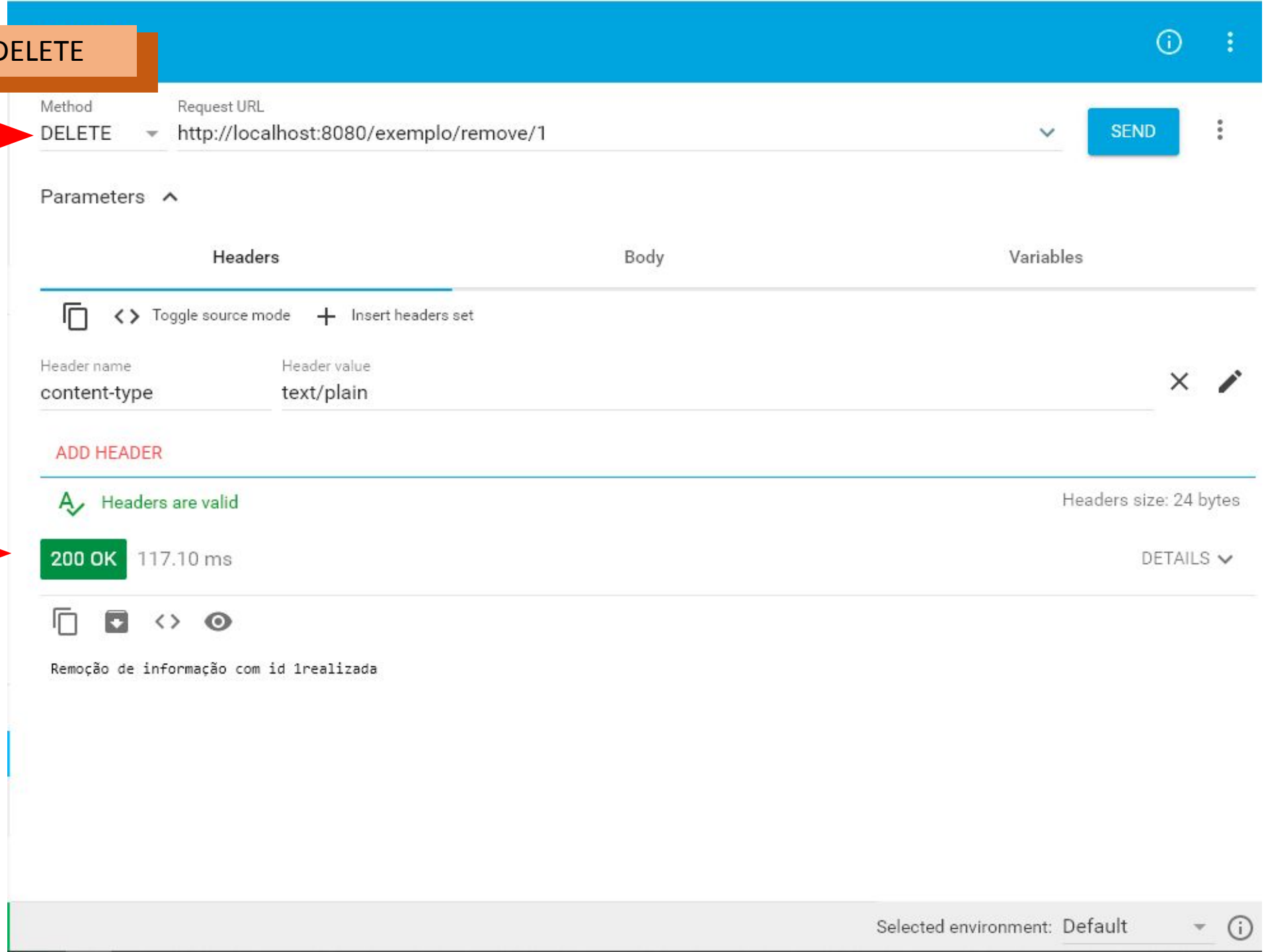
Atenção com a mudança do método HTTP.

Atenção com o formato do dado a ser transmitido. No caso estamos usando JSON.



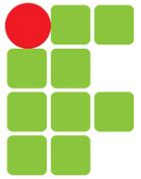
Configuração DELETE

Atenção com a mudança do método HTTP.



Se tudo der certo, uma resposta 200 será retornada.

Criando WebServices com Spring MVC



Nos casos que enviamos um dado JSON, perceba como a estrutura do mesmo deve corresponder com a da classe no Java.

JSON

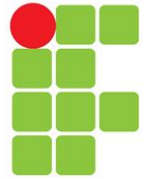
```
{
    "login": "user1",
    "senha": "1234",
}
```

Correspondentes

JAVA

```
class User{
    private String login;
    private String senha;
    public String getLogin() {
        return login;
    }
    public void setLogin(String login) {
        this.login = login;
    }
    public String getSenha() {
        return senha;
    }
    public void setSenha(String senha) {
        this.senha = senha;
    }
}
```

Exercício



Utilizando o projeto de API REST criado na aula, faça as seguintes modificações:

- crie uma interface chamada **UserService** com os métodos:
  - void add(User newUser);
  - User find(String login);
  - boolean remove(String login);
- implemente a interface em uma classe chamada **UserServiceImpl** e defina um **ArrayList<User>** como atributo para armazenar os usuários. Implemente os métodos da interface para trabalhar de acordo com suas ações (nome dos métodos);
- Configure a classe **UserServiceImpl** como um Spring Bean
- Na classe **UserController**, recupere o objeto de **UserServiceImpl** por injeção de dependência com **@Autowired**.
- Altere os métodos que tratam as requisições para utilizar a classe **UserServiceImpl**.

Fontes

SURWASE, Vijay. REST API modeling languages-a developer's perspective. **Int. J. Sci. Technol. Eng**, v. 2, n. 10, p. 634-637, 2016. url:

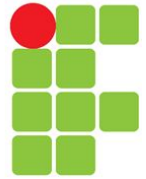
http://www.ijste.org/articles/IJSTEV2I10199.pdf

https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Evolution_of_HTTP

https://www.websiterating.com/resources/http-status-codes-cheat-sheet/

https://restfulapi.net/

Criando WebServices com Spring MVC



Organização do projeto

```
src
└── main
    └── java
        └── com
            └── example
                ├── controller   // Camada de Controller (Web)
                ├── service      // Camada de Serviço (Negócio)
                ├── repository   // Camada de Repositório (Acesso a Dados)
                └── model        // Camada de Model (Entidades)
```